

Design a Rate Limiter - Step 1: Understand the Problem & Establish Design Scope

Rate limiting can be implemented using different algorithms, each with its own pros and cons.

In an interview, clarifying requirements through discussion is critical.

1. Requirement Clarification (Interview Discussion)

Candidate–Interviewer Interaction

Candidate: What kind of rate limiter are we going to design?
Is it client-side or server-side?

Interviewer: Focus on **server-side API rate limiter**.

Candidate: Should rate limiting be based on IP, user ID, or something else?

Interviewer: It should be **flexible** to support multiple throttling rules.

Candidate: What is the scale of the system?

Interviewer: It should handle a **large number of requests**.

Candidate: Will the system be distributed?

Interviewer: Yes.

Candidate: Should it be a separate service or part of application code?

Interviewer: It is **your design choice**.

Candidate: Do we need to notify users when they are rate-limited?

Interviewer: Yes.

2. Final Requirements Summary

Functional Requirements

- Accurately limit excessive requests
- Support multiple throttling strategies (IP, user ID, etc.)
- Inform users when requests are throttled

Non-Functional Requirements

- **Low latency** → should not slow down HTTP responses
- **Memory efficient** → minimal memory usage
- **Distributed system support** → shared across servers/processes
- **High fault tolerance** → failure of one component should not break the system

3. Key Takeaway

Step 1 is about **clarity, not solutioning**

- Ask questions
- Remove ambiguity
- Define scope clearly
- Write assumptions if needed